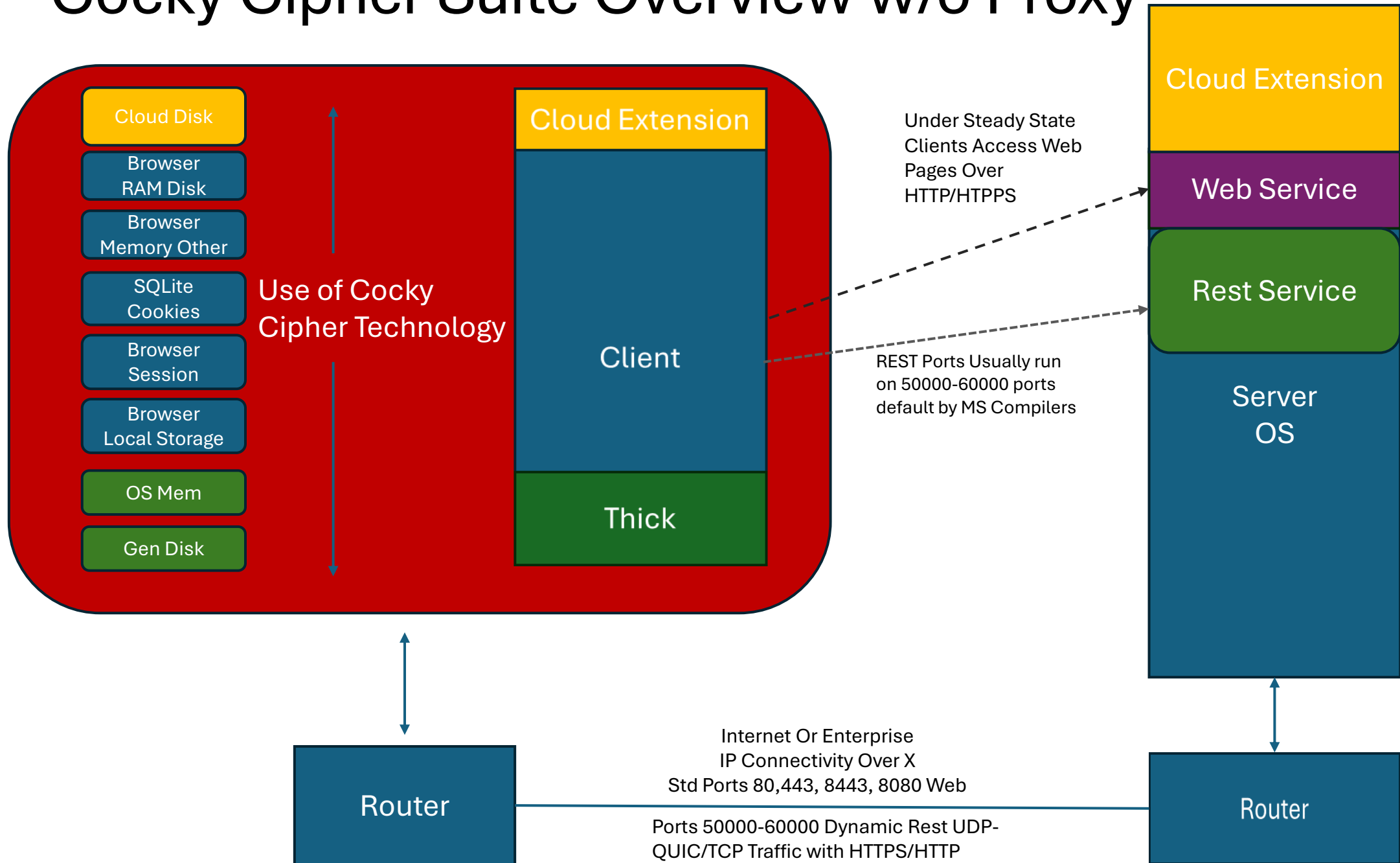


Cocky Cipher Suite Overview w/o Proxy



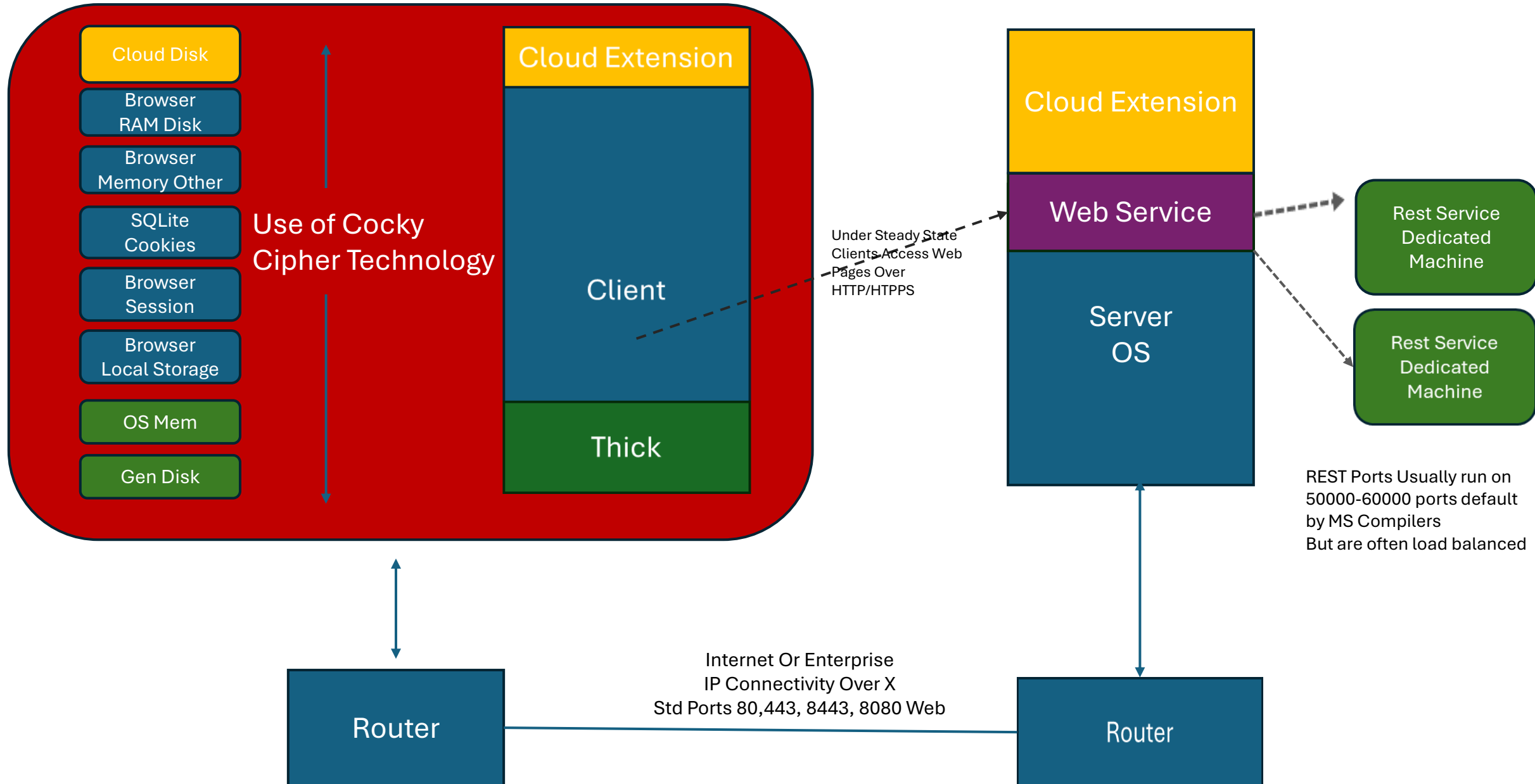
Application Basics

- Web Pages and Web Data Run out of band from each other in a REST model.
- This means there are 4 options – (16 Combinations)
 - With or Without Application Encryption (Clients-Server Other)
 - With or Without Network Encryption for Web. (Web Service)
 - With or Without Network Encryption for Data (REST Service)
 - With or Without Other Network Encryptors/Hardware, etc.
- With a Proxy there are 5 variables (32 Combinations)
 - With Or Without Network Encryption to the Web Server TO URL1 TO WWW
 - With Or Without Network Encryption to the Web Proxy TO URL2 TO REST
 - With Or Without Application Encryption
 - With Or With Other Network Encryptors/Hardware.
 - AND New With or Without SSL From Web Proxy to REST Service.
- If all 4 types of security are enabled using modern tools it would require a state/gov actor and a court order to decrypt client-server traffic in transit or in stored form.

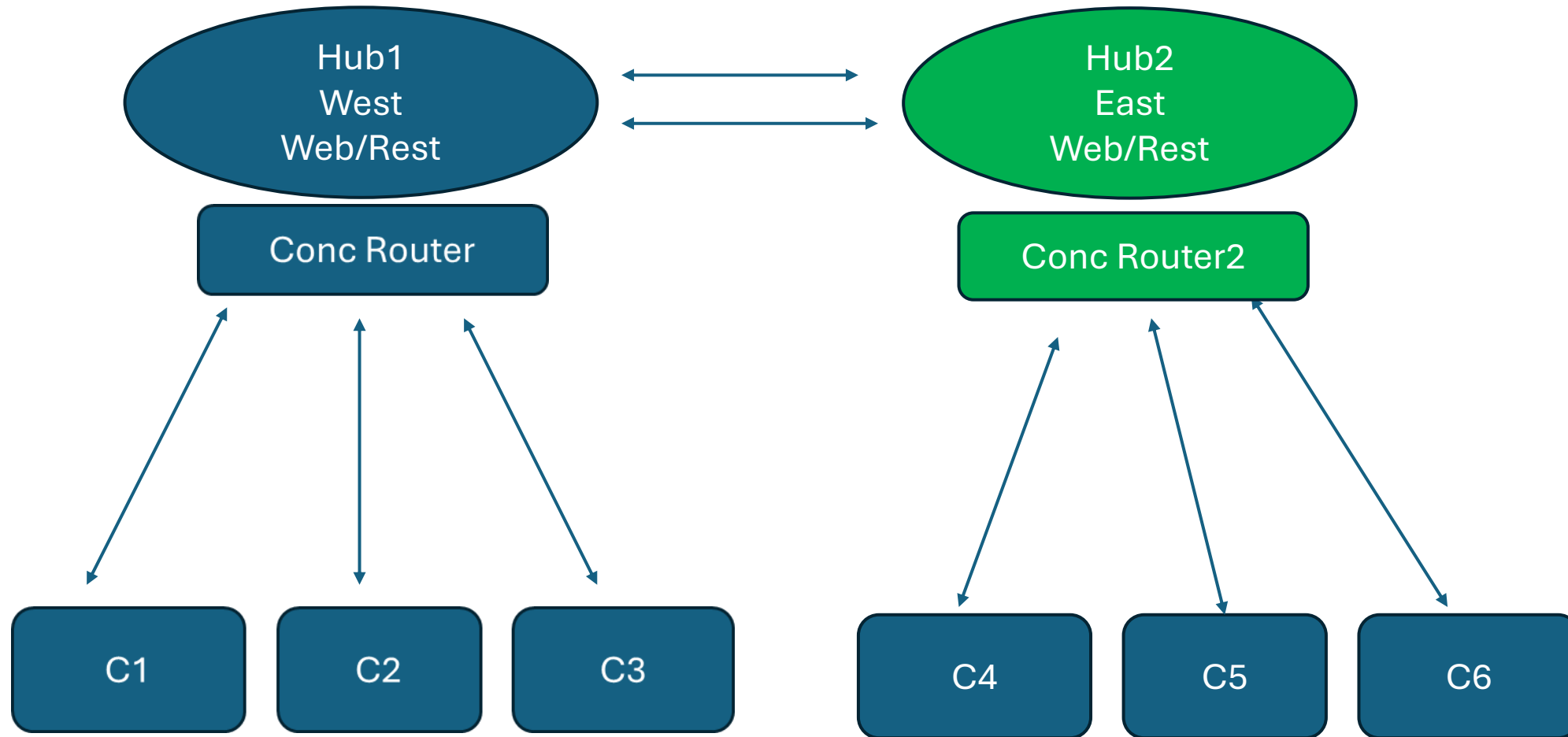
Laws and Compliance

- Most Countries Differentiate Laws For
 - Stored Communications (Already on Disk or in Databases)
 - Network Communications (Real Time Distribution of Streams)
- In the United States
 - Stored Communications Act Governs Requirements for reviewing data on Disk.
 - Real Time Surveillance Seen to be a 4th Amendment violation (See Katz v US) by Supreme Court of US. Prosecuted under Color of Law Statutes.
 - Other State Laws and Provisions.
- Crimes Have To Be Proven Based on Criminal Intent
 - An Encrypted Block of Text even a Caesar Encoded one... establishes intent for a criminal process.
 - Without Application Encryption No Basis for Criminal Prosecution exists. Users can argue they happened on the materials in question by accident.
- Most Large Companies Have a Different Shared Service IT Leader From Business Unit CIOs who set Risk & Compliance policies for their division. That means network and BU's have different constraints.

Cocky Cipher Suite w Proxy – Simple Transport Model

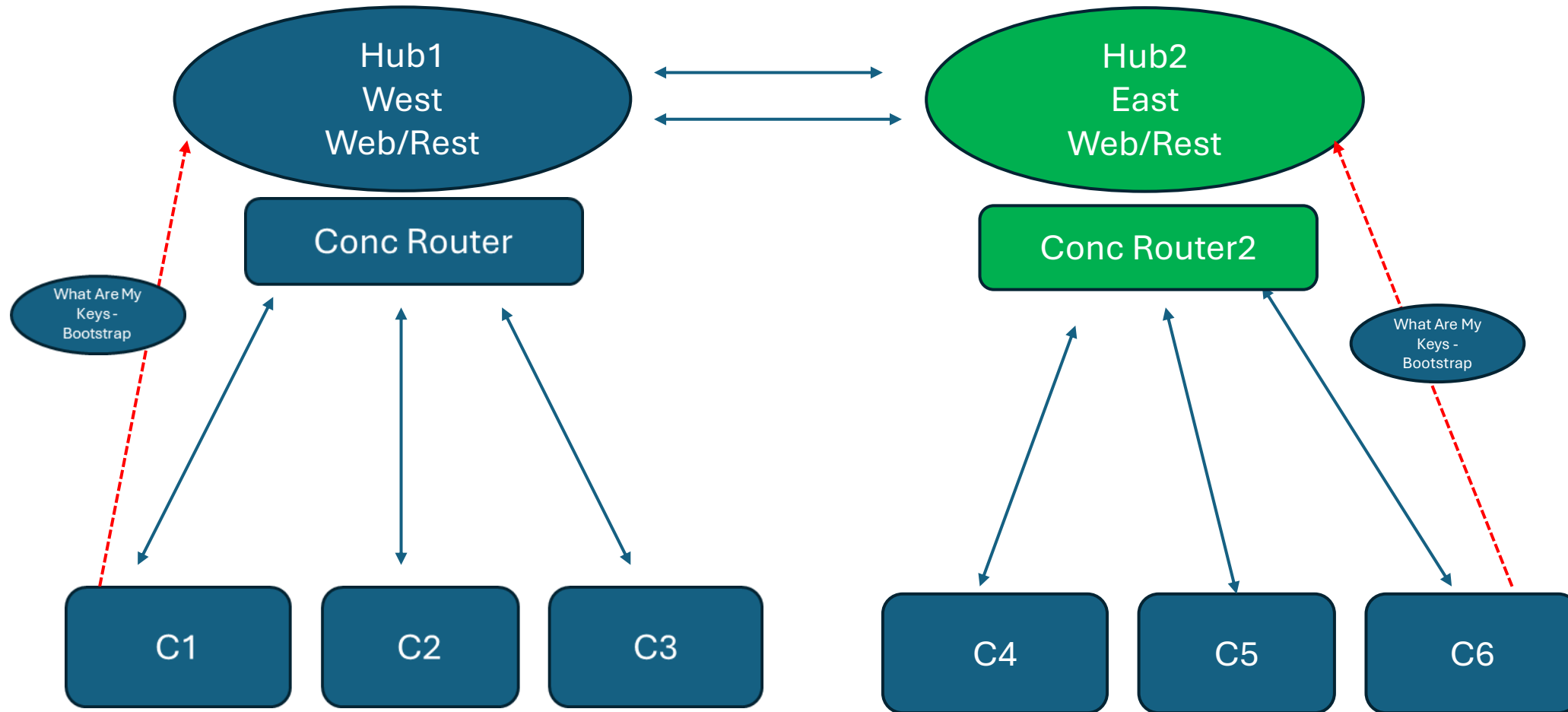


Hub and Spoke Internet Models With Encryption Models



C1-C6 Receive the Same Encryption Keys in Caesarian Models

Encryption - Per Session Models



C1-C6 Receive Different Keys in Per Session Models

Some Perspective... Chrome Defaults to AES128

Copilot Search

Chrome's Default SSL/TLS Cipher Suite

In modern Chrome (as of 2026), there is **no single "default cipher suite" toggle** you can pick – instead, Chrome negotiates a cipher suite with the server during the TLS handshake, and it enforces only modern, secure protocols [www.chromethemer.com +1](#).

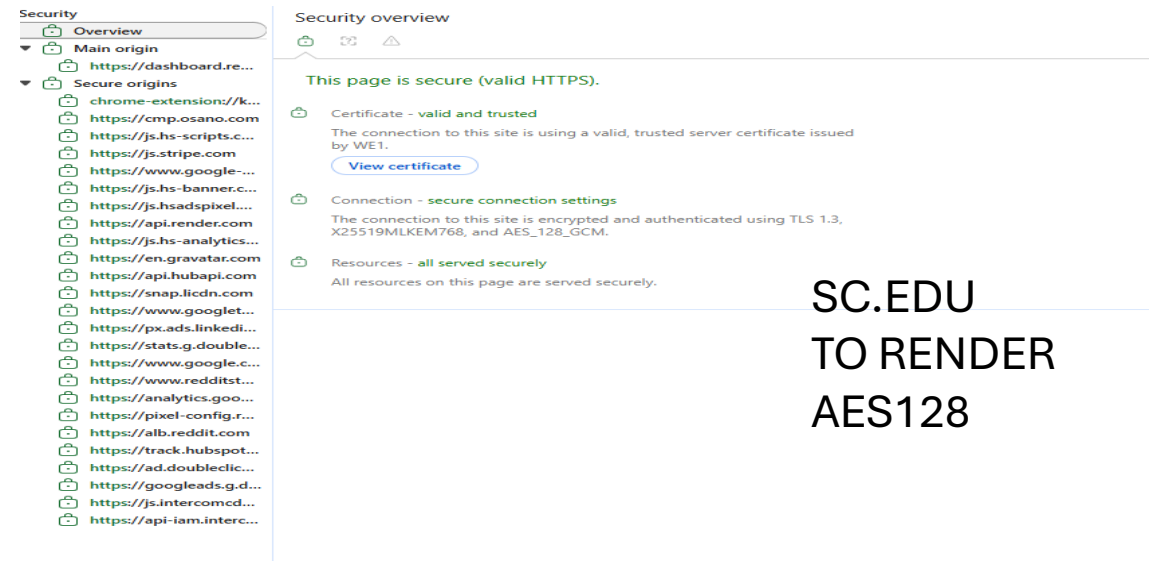
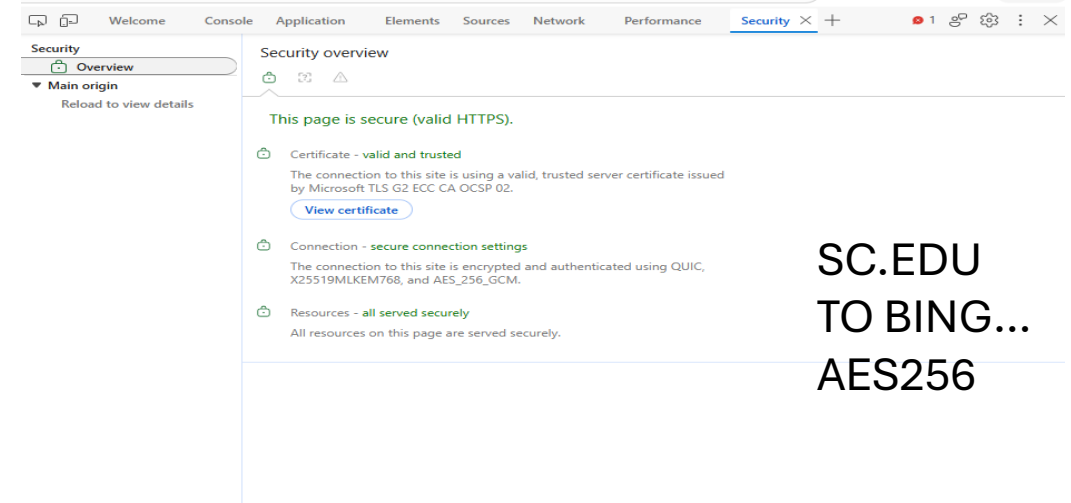
How Chrome Chooses a Cipher Suite

- **Protocol support:** Chrome 152+ only supports **TLS 1.2** and **TLS 1.3**; older versions (TLS 1.0/1.1) are disabled by default [Wboom Designs](#).
- **Key exchange:** Since Chrome 131, the default key exchange is **X25519MLKEM768**, a hybrid of the classic X25519 ECDHE key exchange with a post-quantum KEM (Kyber) for forward secrecy [Wboom Designs](#).
- **Cipher suite selection:** Chrome picks the highest-security cipher suite from the server's list that it supports. This is negotiated in real time, so the "default" depends on the server's configuration.

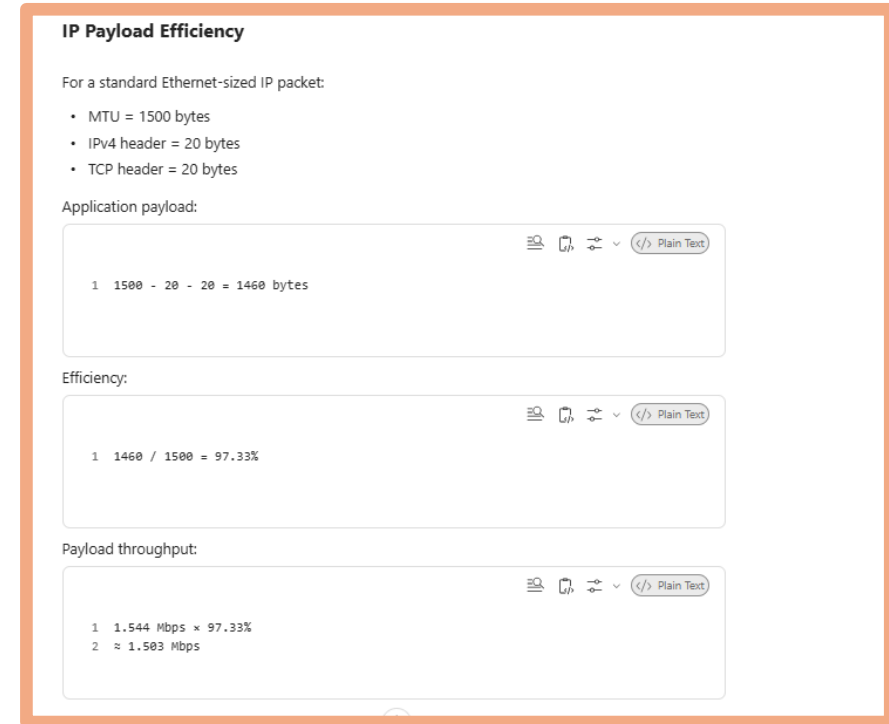
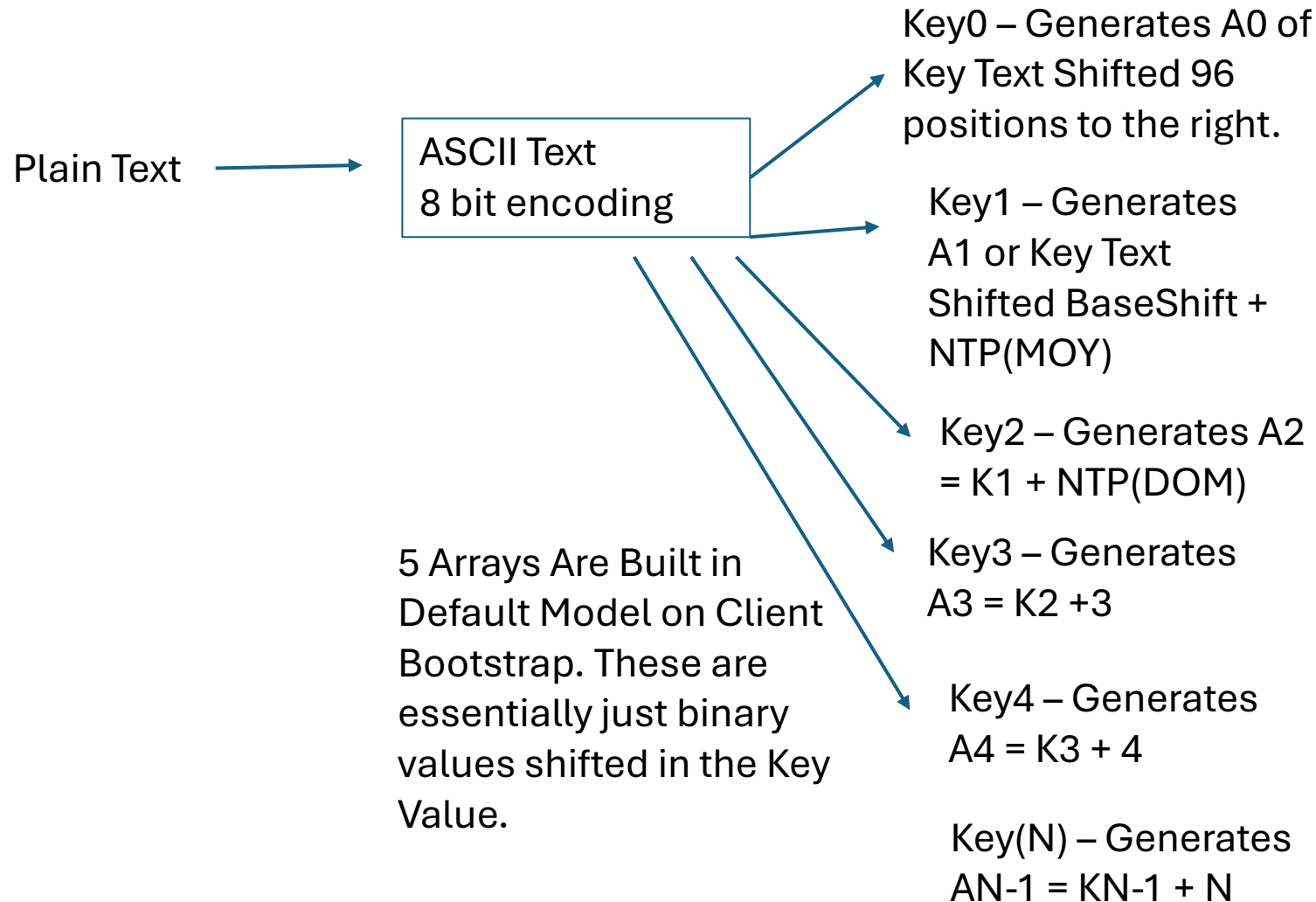
What You Can See

You can inspect the negotiated cipher suite in Chrome's **Developer Tools**:

1. Open DevTools (**Ctrl+Shift+I** / **Cmd+Opt+I**).
2. Go to the **Security** tab.
3. Look at the **Overview** section for:
 - **Protocol:** TLS 1.2 or TLS 1.3
 - **Key Exchange:** e.g., **ECDHE_ECDSA** or **X25519MLKEM768**
 - **Cipher:** e.g., **AES_128_GCM** or **AES_256_GCM**
 - **Hash:** e.g., **SHA256** [Information Security Stack Exchange](#).



Cipher Operations



Plain Text Block Assumed to be 1460 Bytes/characters at Base

Key Max in Default = 96+12+31+3+4+5

The BASICS OF THE COCKEY CIPHER

Key0 is The base shift which defaults to 96 in Binary. Arrays are Now All Present in Memory on Client and Server.



Encrypt(m) – Standard Form.

(m) = Swearingen Is the name of our Engineering Building

Pass 1 we inject a space at each 5th position

(Md1) = Swea ring en is the name of our Engi neer ing Buil ding

(We add a space to every 5th position – could be tuned to 3, 4, 6, any value.

(Md2) = Md1 to ASCII

The 8-bit ASCII values (including every space character as ASCII 32) are:

```
1  83 119 101 97 32 114 105 110 103 32 101 110 32 105 115 32 116 104 101 32 110 97 109
  101 32 111 102 32 111 117 114 32 69 110 103 105 32 110 101 101 114 32 105 110 103
  32 66 117 105 108 32 100 105 110 103
```

Hexadecimal representation:

```
1  53 77 65 61 20 72 69 6E 67 20 65 6E 20 69 73 20
2  74 68 65 20 6E 61 6D 65 20 6F 66 20 6F 75 72 20
3  45 6E 67 69 20 6E 65 65 72 20 69 6E 67 20 42 75
4  69 6C 20 64 69 6E 67
```

(Md3) Final – Codes based on their position are remapped to the encryption matrix in Linear time. A simple loop from (i to 1460). This is two assembler instructions or perhaps 1 and requires no multiplication. $O(n)$.

Eg 83 -> Key{0, 96+83}

Eg 119 -> Key(1, 96+119+MOY) = Key1

Ed 101 -> Key(2, Key1 + DOM) = Key2

This is matrix shifted algorithm similar to the Enigma or Vigenère's Array of Substitutions Model.

Encrypt(m) – Advanced Form

(m) = Swearingen Is the name of our Engineering Building

Pass 1 we inject a space at each 5th position

(Md1) = Swea ring en is the name of our Engi neer ing Buil ding

(We add a space to every 5th position – could be tuned to 3, 4, 6, any value.

(Md2) = Md1 to ASCII

The 8-bit ASCII values (including every space character as ASCII 32) are:

```
1  83 119 101 97 32 114 105 110 103 32 101 110 32 105 115 32 116 104 101 32 110 97 109
   101 32 111 102 32 111 117 114 32 69 110 103 105 32 110 101 101 114 32 105 110 103
   32 66 117 105 108 32 100 105 110 103
```

Hexadecimal representation:

```
1  53 77 65 61 20 72 69 6E 67 20 65 6E 20 69 73 20
2  74 68 65 20 6E 61 6D 65 20 6F 66 20 6F 75 72 20
3  45 6E 67 69 20 6E 65 65 72 20 69 6E 67 20 42 75
4  69 6C 20 64 69 6E 67
```

(Md3) Final – Codes based on their position are remapped to the encryption matrix in Linear time. A simple loop from (i to 1460). This is two assembler instructions or perhaps 1 and requires no multiplication. $O(n)$.

Eg 83 -> Key{0, A0{83}}

Eg 119 -> Key(1, A0{119})

Ed 101 -> Key(2, A2{101})

In this model the N arrays of 8bit values are sent over the wire.

There is no shift. Just an array of binary values. This is a random 8 bit code block.

Engima (A Four Wheeled Device)



https://en.wikipedia.org/wiki/Enigma_machine

The Array is Cached in Memory Before Encryption. This is like setting the dials of the Engima Machine. The Encryption and Decryption are in Linear Time.

The Arrays of Text We have built in Memory Are More Secure than Enigma Machine which used only 26 Latin Letters. But had 26^4 permutations possible.

We are using (N) number of keys with 5 being the base on 8bit codes which are 256^5 permutations. 10x larger than the original.

Whats different? (1) our first pass, breaks words into smaller sub-words. (2) CockyCiphers can be modified per stream making encryption all but impossible except for Nation States. (3) Advanced Versions using 16bit or 32bit multipliers (4) Possible to scramble all five arrays randomly per stream using Network Streams from REST Bootstrap Making the Cipher a Matrix Cipher.

Classifying Our Algorithm?

- 1) Substitution Cipher -> Fixed Shift. (Letters – 1 Permutation)
- 2) Substitution Cipher -> Random Shifts -> Monoalphabetic Substitution Cipher-> CG Melon (26! -1)
- 3) Substitution Cipher -> Adv Random Shifts -> ASCII Based Substitution Cipher -> (256! -1) [IE JUST ASCII]
- 4) Array Block Cipher -> Binary Matrix Variant – VignereCipher+ -> Array Substitution Cipher (256!) exp 5.
- 5) Enigma Radial Cipher -> 4 Wheel Encoding Machine -> Array Substitution Cipher(26!) exp 4.

The Vigenère cipher had a starting single key. Our has 5 that include day mutators.

The Vigenère cipher used 26 characters, were are using ASCII shifted values with 256 values.

Cocky Cipher Suite

Any Cipher we can code that can provide PCI, and HIPAA Compliance.

This is essentially a requirement to eliminate all observable customer information for payments or otherwise on disk and in transit. That's more than a discussion of SSL technology.

Caesar-3, Caesar-7, AES, 3DES, CockyDefault, CockyDef2(Over Network Scrambled), and CockyExtreme(Stream) which goes to 16bit codes are what I am planning to support client to server.

To support a suite of Ciphers from client to server:

- a) Must have a Bootstrap API
- b) Must have an Auth API
- c) Must have a Time API to verify for thick clients and servers they have ntp enabled and working.

The End Goal... Clients Running Cocky Analytics Securely.... And Storage Areas Encrypted

CockyAnalytics by Citi

Sign in



STOCK ANALYZER

Market intelligence, simplified.

Sign in to access your portfolio and analytics.

Get started

MARKET CLOSED
[SAVED]

\$192.45 +0.63%

GOOGL \$174.99 +0.29%

META \$558.12 +1.32%

SPY \$547.20 +0.15%

QQQ \$468.87 +0.41%

18:30:49 ET

Welcome Console Application Elements Sources Network Performance Security 60 10

Security

Overview

▼ Main origin

▼ Unknown / canceled

https://query1.financ...

Security overview

This page is secure (valid HTTPS).

Certificate - valid and trusted

The connection to this site is using a valid, trusted server certificate issued by WE1.

View certificate

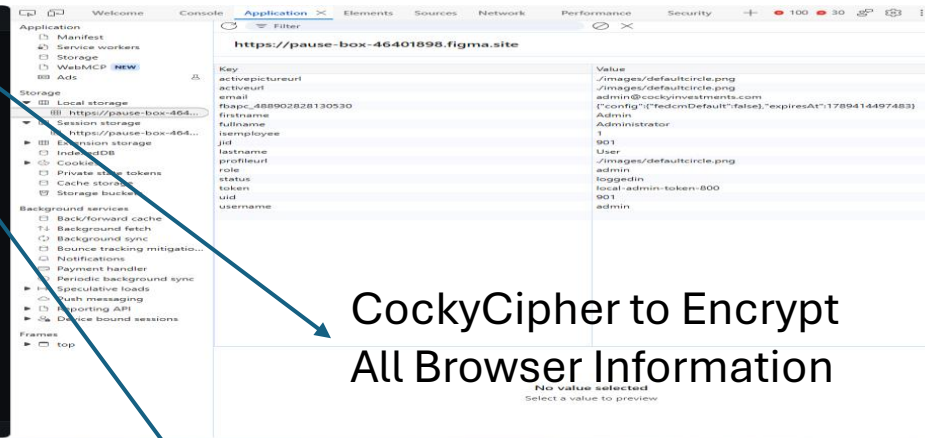
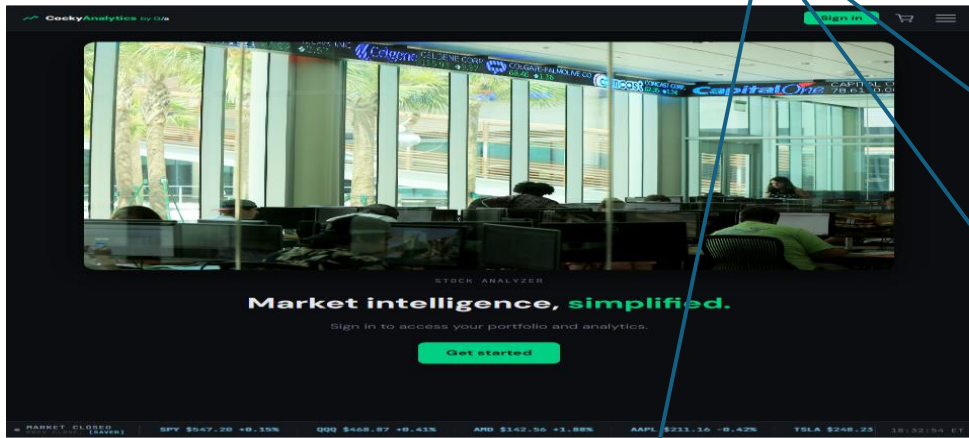
Connection - secure connection settings

The connection to this site is encrypted and authenticated using TLS 1.3, X25519MLKEM768, and AES_128_GCM.

Resources - all served securely

All resources on this page are served securely.

And All PCI Data Masked



CockyCipher to Encrypt
All Browser Information

